AAS in AI Software Engineering

FE101

Spread operator (...)

Today

Welcome to your next lesson: Spread operator (...)

I'm getting your personalized lesson ready — based on what you've learned, what's ahead, and how you learn best.

Step by step, you’re moving forward.

Let’s start with a quick idea you already know from Python 🔁

In JavaScript, some values can be changed in place (like Python lists/dicts), and some cannot (like Python ints/strings).

Mutable → you can change the inside after it’s created
Immutable → you cannot change the inside; you can only replace the whole value

Very soon we’ll connect this to the spread operator ..., but first I want this base clear.

Does this mapping match your current mental model?

Numbers, strings, booleans → immutable
Arrays, objects → mutable

Answer in your own words: does that feel right to you, or is any part of it fuzzy?

Yes, that matches my understanding

Great 👍 that's exactly what we need.

Next step: let's see mutability in action with a tiny example.

In the code editor, type and run this:

```
Javascript
const score = 10;
const scores = [10, 20, 30];

scores[0] = 99;
// score = 99; // leave this line commented out

console.log("score:", score);
console.log("scores:", scores);
```

Don't tell me the output yet.

First: in your head, decide what you expect to see for score and for scores.

Then run it, and tell me: did the result match what you expected, or was anything surprising? 🎯

yes it matched

Nice 💪 That confirms your intuition is solid.

Now I want to show you why copying arrays/objects can be tricky — this is the key to understanding the spread operator.

In the editor, type and run this:

```
Javascript
const original = [1, 2, 3];
const alias = original;   // looks like a copy, but is it?

alias[0] = 999;

console.log("original:", original);
console.log("alias:", alias);
```

Again, first predict in your head:

Will changing alias[0] also change original[0], or not?

Then run it and tell me which happened:

both arrays changed
only alias changed
only original changed
it worked as i expected const alias = orginal doesnt make a new copy the same as python, it makes alias point to the actual inmemory copy of orfginal so modif one modifies both

Exactly right 👌 — perfect Python-to-JS transfer.

const alias = original; → no new array, just another name (alias) for the same array in memory
So alias[0] = 999; changes the one shared array, and both logs show the change

This “shared reference” is why we need the spread operator ... to make a real copy.

Next step: let’s create a true copy with spread and compare.

In the editor, write and run this:

Javascript
```
const original = [1, 2, 3];
const copy = [...original];   // using spread

copy[0] = 999;

console.log("original:", original);
console.log("copy:", copy);
```
After you run it, tell me:

Did changing copy[0] leave original[0] alone this time?

yes originsl stayed uncjaged

Perfect 🎯

So now we’ve seen:

const alias = original → same array, just two names (change one, both reflect it)
const copy = [...original] → new array with the same top-level values (change one, the other stays the same)

That’s the core idea of using ... for arrays.

Let’s do the same kind of thing with an object now (very similar to a Python dict).

In the editor, try this:

Javascript
```
const user = { name: "Ron", level: 1 };
const twin = user;   // alias, not a copy

twin.level = 99;

console.log("user:", user);
console.log("twin:", twin);
```
Run it, then tell me in words what you saw:

Did changing twin.level also change user.level?

both chnaged

Right — same story as arrays ✅

twin and user are just two names pointing to the same object in memory
Changing twin.level changes the shared object, so user.level appears changed too

This is reference-based assignment: objects and arrays are shared unless you explicitly copy them.

Now let’s do the spread version for objects so you have the contrast.

In the editor, run this:

```
Javascript
const user = { name: "Ron", level: 1 };
const clone = { ...user };   // spread copy

clone.level = 99;

console.log("user:", user);
console.log("clone:", clone);
```

Run it, then tell me: did changing clone.level leave user.level alone this time?

user remained unchanged

Nice, you're tracking this really well 🔥

So far you've seen:

Arrays/objects are mutable and assigned by reference
Spread ... gives you a new top-level array/object, so changes to the copy don't hit the original

Now I want to show you the one subtle catch: spread is a shallow copy, not a deep one.

In the editor, try this nested example:

```
Javascript
const original = [[1, 2], [3, 4]];
const copy = [...original];

copy[0][0] = 999;

console.log("original:", original);
console.log("copy:", copy);
```

Before you run it, pause and predict:
Will original change, copy change, or both?
Then run it and tell me what actually happened.
they both changed i would have thought they would have been unique

Yeah, this is the sneaky part 😄

What happened:

copy = [...original] made a new outer array
But the inner arrays ([1, 2] and [3, 4]) were not copied — they were just references reused
So copy[0][0] = 999 actually mutated that shared inner array, which both original[0] and copy[0] point to

Visually:

original → 🔗 innerArrayA, 🔗 innerArrayB
copy → 🔗 innerArrayA, 🔗 innerArrayB (same inner references)

So: spread gives a shallow copy — only the top level is new.

Quick check, in your own words (short is fine):

How would you describe the difference between:

```
const alias = original;
const copy = [...original];
```

especially when original is not nested vs when it is nested?

1 creates a secdond name for the same date
2 creates a new dataset , using the original as the template for its values, but its only one layer deep, nested arrays are still the same as the original

That's an excellent summary, Ron 💯

alias → just a second name for the same data
spread copy → new top-level container, but nested arrays/objects are still shared

That's exactly the mental model you want for mutable vs immutable, reference assignment, and spread shallow copy ✅

Before we wrap this lesson, I'd like you to do a small "all-in-one" demo in the editor to lock it in:

Build a short script that shows all of these in order:

A primitive (like a number) staying unchanged when you “copy” it
An array alias where changing one changes both
An array spread copy where changing the copy does not change the original
A nested array spread copy where a nested change does affect the original

Use console.log after each part so it’s clear what happened.

When you’re done, click Submit code, and I’ll review it with you 👀

Javascript

```
const singular_arr = [1, 2, 3, 4];
const spread_arr = [4, 5, 6, 7];
const nested_arr = [[1, 2], [3, 4]];

const copy1 = singular_arr;
copy1[0] = 99;
console.log("copy1", copy1);
console.log("singular_arr", singular_arr);

const copy2 = [...spread_arr];
copy2[0] = 99;
console.log("copy2", copy2);
console.log("spread_arr", spread_arr);

const copy3 = [...nested_arr];
copy3[0][0] = 999;
console.log("original:", copy3);
console.log("copy:", nested_arr);
```

Love this, Ron — clean and exactly on target 🎯

Quick walkthrough of what your code shows:

1️⃣ Alias with a plain array


```
const singular_arr = [1, 2, 3, 4];
const copy1 = singular_arr;
copy1[0] = 99;
```

Output shows:

```
copy1 → [99, 2, 3, 4]
singular_arr → [99, 2, 3, 4]
```

✅ Demonstrates reference aliasing: both names point to the same array, so mutating one mutates both.

2️⃣ Spread copy with a flat array

```
const spread_arr = [4, 5, 6, 7];
const copy2 = [...spread_arr];
copy2[0] = 99;
```

Output shows:

```
copy2 → [99, 5, 6, 7]
spread_arr → [4, 5, 6, 7]
```

✅ Demonstrates spread as a safe top-level copy: new array, original untouched.

3️⃣ Shallow copy with a nested array

```
const nested_arr = [[1, 2], [3, 4]];
const copy3 = [...nested_arr];
copy3[0][0] = 999;
console.log("original:", copy3);
console.log("copy:", nested_arr);
```

Output shows both:

```
original: [ [ 999, 2 ], [ 3, 4 ] ]
copy:     [ [ 999, 2 ], [ 3, 4 ] ]
```

✅ Demonstrates shallow copy limitation: outer array is copied, inner arrays are still shared references, so nested mutation affects both.

Only tiny naming thing: your logs

```
console.log("original:", copy3);
console.log("copy:", nested_arr);
```

have the labels flipped — but logically you used them as original (nested_arr) and copy (copy3), and the behavior is correct, so I know you understand what's going on.

From everything you've done and explained:

You understand mutable vs immutable in JS
You get reference-based assignment and aliasing
You can use spread `...` to make shallow copies and predict when originals change vs stay the same

That's exactly what this lesson was aiming for ✅

If you want, I can give you a quick one-paragraph summary to keep as a note, or we can roll straight into the next lesson. What do you prefer?

How would you rate this lesson?

sure

Here's a compact summary you can keep handy 📌

1. Mutable vs immutable

Immutable (cannot change in place):
numbers, strings, booleans

```
let a = 10;
let b = a;
b = 20;     // a stays 10
```

Mutable (can change in place):
arrays, objects

```
const arr = [1, 2];
arr[0] = 99;   // arr becomes [99, 2]
```

2. Reference-based assignment (aliasing)

Arrays/objects are assigned by reference:

Javascript
```
const original = [1, 2, 3];
const alias = original;   // no copy!

alias[0] = 999;

console.log(original); // [999, 2, 3]
console.log(alias);    // [999, 2, 3]
```
Both variables point to the same array in memory.

3. Spread for shallow copies

Use ... to make a new top-level array or object:

Javascript
```
const nums = [1, 2, 3];
const copy = [...nums];

copy[0] = 999;

console.log(nums); // [1, 2, 3]
console.log(copy); // [999, 2, 3]
```
Javascript
```
const user = { name: "Ron", level: 1 };
const clone = { ...user, level: 5 };  // also lets you override fields

console.log(user);  // { name: "Ron", level: 1 }
console.log(clone); // { name: "Ron", level: 5 }
```
4. Shallow copy limitation (nested data)

Spread only copies one level deep:

Javascript
```
const nested = [[1, 2], [3, 4]];
const shallowCopy = [...nested];

shallowCopy[0][0] = 999;

console.log(nested);      // [ [999, 2], [3, 4] ]
```

```
console.log(shallowCopy); // [ [999, 2], [3, 4] ]
New outer array ✅
Inner arrays still shared ❌
If you’d like next, we can either:


compare this mentally to how Python lists/dicts behave, or
move on to the next JS topic.

What are you in the mood for?


const original = [1, 2, 3];
const copy = [...original];   // using spread

copy[0] = 999;

console.log("original:", original);
console.log("copy:", copy);


const user = { name: "Ron", level: 1 };
const clone = { ...user };   // spread copy

clone.level = 99;

console.log("user:", user);
console.log("clone:", clone);


const singular_arr = [1, 2, 3, 4];
const spread_arr = [4, 5, 6, 7];
const nested_arr = [[1, 2], [3, 4]];

const copy1 = singular_arr;
copy1[0] = 99;
console.log("copy1", copy1);
console.log("singular_arr", singular_arr);

const copy2 = [...spread_arr];
copy2[0] = 99;
console.log("copy2", copy2);
console.log("spread_arr", spread_arr);

const copy3 = [...nested_arr];
copy3[0][0] = 999;
console.log("copy3:", copy3);
```

```
  console.log("nested_arr:", nested_arr);

```